modules require a USB to TTL converter and a 3.3-volt stable power source to interact with the computer. Hence, we can use a complete ESP8266 board like NodeMCU or Adafruit Feather HUZZAH, which have all the necessary components on the same board.

Figure 5: NodeMCU

To flash the MicroPython firmware binary on the ESP8266 board, a handy command line tool known as *esptool* can be used. It can be downloaded directly from the Python package manager using the following commands:

```
sudo pip install esptool
Check for the serial port after connecting the board it is /
dev/tty.SLAB_USBtoUART in our case
ls /dev/tty* can help you identify it.
esptool.py --port "serial_port" erase_flash
esptool.py --port "serial_port" --baud 460800 write_flash
--flash_size=detect 0 "firmware_file"
```

Here, the firmware file *.bin* should be kept in the current working directory:

```
iAyan:~ iAyan$ sudo esptool.py --port /dev/tty.SLAB_USBtoUART
erase_flash
esptool.py v1.2.1
Connecting...
Running Cesanta flasher stub...
Erasing flash (this may take a while)...
Erase took 10.8 seconds
iAyan:~ iAyan$ esptool.py --port /dev/tty.SLAB_USBtoUART
--baud 460800 write_flash --flash_size=detect 0 esp8266-
20161110-v1.8.6.bin
esptool.py v1.2.1
Connecting...
Auto-detected Flash size: 32m
Running Cesanta flasher stub...
Flash params set to 0x0040
Writing 569344 @ 0x0... 569344 (100 %)
Wrote 569344 bytes at 0x0 in 14.1 seconds (324.1 kbit/s)...
Leaving...
iAyan:~ iAyan$
```

Using the serial REPL (Read Evaluate Print Loop)

Just like flashing MicroPython to the board, its REPL can be accessed over the serial port. Simply connect your MicroPython compatible board, the ESP8266 board in this case, and it will mount as a Serial Device (COMMxx in Windows and */dev/ttySiLabs* in OSX and Linux). Use any serial emulator like Putty (on Windows), minicom or screen on Linux/OSX to access the serial REPL, using the baud rate 115200, as follows:

```
sudo screen /dev/tty.SLAB_USBtoUART 115200
```

or

```
sudo minimum -D /dev/ttySLAB_USBtoUART
```

In Windows use Putty, for which the serial to be used is COMMxx (xx=COMM port number under Device Manager) and baud rate is 115200.

If everything goes well, you will see the familiar Python prompt, at which point you can write Python commands and view the immediate outputs. Type 'print ("Hello World!")', and yes, it's the same old Python that we love.

```
sudo screen /dev/tty.SLAB_USBtoUART 115200
"press enter"
>>> print("Hello, MicroPython!")
Hello, MicroPython!
>>> 2+5
7
>>>
```

Using WebREPL

One unique feature of MicroPython on the boards which support networking (like ESP8266) is a WebREPL (read-evaluate-print loop, like a Python 'command line') that is accessible through a Web page. Instead of using a serial connection to the board, you can run Python code directly from your browser in a simple terminal. You don't even need to connect the board to a Wi-Fi network; it can create its own network, which you can use to access the WebREPL! The new releases of MicroPython come with WebREPL disabled by default; so use the following commands to enable it over serialREPL:

```
>>> import webrepl_setup
WebREPL daemon auto-start status: disabled
Would you like to (E)nable or (D)isable it running on boot?
(Empty line to quit)
> E
To enable WebREPL, you must set password for it
New password: python
Confirm password: python
Changes will be activated after reboot
Would you like to reboot now? (y/n) y
```

Once it's enabled, ESP8266 will start in AP mode and create a hotspot. You can connect to it with the password *micropythoN* and access WebREPL using the IP *ws://192.168.4.1:8266/*. Enter the password that you've set earlier. The WebREPL client can be accessed by going to *http://micropython.org/webrepl/* or downloading it via GitHub (*https://codeload.github.com/micropython/webrepl/zip/master)*.